Numpy

```
Start coding or generate with AI.
```

- 1D array

**Deffination ** NumPy (short for Numerical Python) is an open-source Python library used for working with large, multi-dimensional arrays and matrices, along with a massive collection of high-level mathematical functions to operate on these arrays.

Why Use NumPy?

Faster: It uses contiguous memory, making it much quicker for the CPU to access.

Memory Efficient: It uses less space than Python lists.

Powerful: It allows for "vectorization," meaning you can perform an operation on an entire array at once without writing a for loop.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Basics & Array Creation The core of NumPy is the ndarray (n-dimensional array). Unlike Python lists, these are memory-efficient and allow for vectorized operations.

```
**1D array**
```

```
import numpy as np

d=np.array([1, 2, 3])

print(d)
```

```
[1 2 3]
```

Create a 3x4 array of zeros

```
import numpy as np
w=np.zeros((3, 4))
print(w)
```

```
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

Create a 2x2 array of ones

```
import numpy as np
s=np.ones((2, 2))
print(s)
```

```
[[1. 1.]
 [1. 1.]]
```

Double-click (or enter) to edit

Create a 2x2 array of ones

```
import numpy as np
w=np.full((2, 2), 7)
print (w)
```

```
[[7 7]
 [7 7]]
```

Create a 4x4 Identity matrix

```
import numpy as np
r=np.eye(4)
print (r)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

Array from 0 to 10 with a step of 2

```
import numpy as np
r=np.arange(0, 10, 2)
print (r)
```

```
[0 2 4 6 8]
```

evenly spaced values between 0 and 1

```
import numpy as np
np.linspace(0, 1, 5)
print (r)
```

```
[0 2 4 6 8]
```

NumPy array (called an ndarray) is a grid of values, all of the same type, indexed by a tuple of non-negative integers.

1. **The Anatomy of an Array** To understand NumPy, you have to understand its three main levels:

1D Array (Vector): A single row of data. Indexing is simple: arr[0].

2D Array (Matrix): Data arranged in rows and columns. Indexing uses arr[row, col].

3D Array (Tensor): Multiple matrices stacked together. Think of it like a book where each page is a 2D matrix.

** Shape of array**

```
import numpy as np

arr = np.arange(12) # [0, 1, 2, ..., 11]
print(arr.shape)
```

```
(12,)
```

**size of the array **

```
import numpy as np
arr = np.arange(12) # [0, 1, 2, ..., 11]
print(arr.size)
```

```
12
```

Datatype of array

```
import numpy as np
arr = np.dtypes
print(arr)
```

```
<module 'numpy.dtypes' from '/usr/local/lib/python3.12/dist-packages/numpy/dtypes.py'>
```

** Reshape of array **

```
import numpy as np

# Let's start with a 1D array of 6 elements
data = np.array([1, 2, 3, 4, 5, 6])

# Method 1: Calling reshape on the array itself (Most common)
new_arr = data.reshape(2, 3)

# Method 2: Using the np.reshape() function
# Syntax: np.reshape(array, (new_shape))
new_arr = np.reshape(data, (3, 2))

print(new_arr)
```

```
[[1 2]
 [3 4]
 [5 6]]
```

Indexing and Slicing are how you "zoom in" on the specific data you need. Since NumPy arrays can have multiple dimensions, the syntax is slightly more powerful than standard Python lists.

```
import numpy as np

arr = np.array([
    [10, 11, 12],  # Row 0
    [20, 21, 22],  # Row 1
    [30, 31, 32]   # Row 2
])

# arr[0, 1] selects:
# Row index 0 -> [10, 11, 12]
# Column index 1 -> 11

print(arr[0, 1])
# Output: 11
```

```
11
```

Select the entire second column.

```
import numpy as np

arr = np.array([
    [10, 11, 12],  # Row 0
    [20, 21, 22],  # Row 1
    [30, 31, 32]   # Row 2
])

# arr[0, 1] selects:
# Row index 0 -> [10, 11, 12]
# Column index 1 -> 11

print(arr[0 :1])
# Output: 11
```

```
[[10 11 12]]
```

Double-click (or enter) to edit

Select rows 1 and 2, and all columns.

```
# To select Row 1 and Row 2 (up to but not including 3), and all columns:
print(arr[1:3, :])

# Output:
# [[20 21 22]
```

```
#  [30 31 32]]
```

```
[[20 21 22]
 [30 31 32]]
```

Boolean Indexing

```
import numpy as np

arr = np.array([1, 6, 2, 8, 3, 10])

# 1. Create the mask
mask = arr > 5
print(mask)
# Output: [False  True False  True False  True]

# 2. Apply the mask
result = arr[mask]
print(result)
# Output: [6 8 10]
```

```
[False  True False  True False  True]
[ 6  8 10]
```

Double-click (or enter) to edit

Mathematical Operations One of NumPy's superpowers is Broadcasting, allowing operations between arrays of different (but compatible) shapes.

** Addition**

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Both are equivalent:
result = np.add(a, b)
# result = a + b

print(result) # Output: [5 7 9]
```

```
[5 7 9]
```

**Subtruction **

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Both are equivalent:
result = np.subtract(a, b)
# result = a - b

print(result)
```

```
[-3 -3 -3]
```

Multiply

Double-click (or enter) to edit

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Both are equivalent:
result = np.multiply(a, b)
# result = a *b
```

```
print(result)
```

```
[ 4 10 18]
```

Double-click (or enter) to edit

Matrix multiplication

```
# A is (2x3), B is (3x2)
A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])

# Method 1: The modern @ operator (Recommended)
result = A @ B

# Method 2: The np.matmul() function
result = np.matmul(A, B)

# Method 3: The np.dot() function
result = np.dot(A, B)

print(result)
# Output:
# [[ 58  64]
#  [139 154]]
```

```
[[ 58  64]
 [139 154]]
```

Statistics & Aggregation In data analysis, Statistics and Aggregation are used to compress large arrays into meaningful summaries (like a single average or a total sum). NumPy is exceptionally fast at this because it uses highly optimized C and Fortran code under the hood.

axis=0 (The Column-Wise Operation) When you set axis=0, you are telling NumPy to perform the action down the columns. Think of it as "collapsing the rows" until only one row remains. **axis=1 (The Row-Wise Operation)** When you set axis=1, you are telling NumPy to perform the action across the rows. Think of it as "collapsing the columns" until only one column remains.

Python

```
import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60]])

# axis=0: 10+40, 20+50, 30+60
column_sum = np.sum(arr, axis=0)

print(column_sum)
# Output: [50, 70, 90]
```

```
[50 70 90]
```

```
# axis=1: 10+20+30, 40+50+60
row_sum = np.sum(arr, axis=1)

print(row_sum)
# Output: [60, 150]
```

```
[ 60 150]
```

**Mean of the array **

```
import numpy as np

# A 2x3 array (2 rows, 3 columns)
arr = np.array([[10, 20, 30],
                [40, 50, 60]])
```

```
# axis=1: (10+20+30)/3 and (40+50+60)/3
row_means = np.mean(arr, axis=1)

print(row_means)
# Output: [20.0, 50.0]
```

```
[20. 50.]
```

Median of the array

```
import numpy as np

prices = np.array([10, 20, 30, 40, 1000]) # 1000 is an outlier

print(np.mean(prices))
# Output: 220.0 (The 1000 pulled the average way up!)
```

```
220.0
```

Standard Deviation

```
import numpy as np

# Scenario A: Consistent scores
group_a = np.array([80, 82, 81, 79, 80])
print(np.std(group_a))
# Output: ~1.0 (Very low spread)

# Scenario B: High variance scores
group_b = np.array([50, 100, 20, 150, 80])
print(np.std(group_b))
# Output: ~46.0 (Huge spread!)
```

```
1.019803902718557
44.27188724235731
```

Basic Min & Max By default, these look at the entire array and return the single smallest or largest value **Data Normalization :**A common use case for these in Machine Learning is Min-Max Scaling, which squashes all your data between 0 and 1. You can add this "Pro Formula" to your LinkedIn note:

$$\text{scaled\_arr} = \frac{\text{arr} - \text{np.min(arr)}}{\text{np.max(arr)} - \text{np.min(arr)}}$$

Python

```
import numpy as np

arr = np.array([15, 30, 5, 45, 20])

print(np.min(arr)) # Output: 5
print(np.max(arr)) # Output: 45
```

```
5
45
```

Standard Deviation** **bold text** is the "consistency meter." While the mean tells you the center of your data, the standard deviation tells you how spread out the numbers are from that center.

Low Standard Deviation: The numbers are all very close to the mean (consistent data).

**High Standard Deviation: **The numbers are spread far apart (volatile or diverse data).

```
import numpy as np

# Scenario A: Consistent scores (Low Spread)
group_a = np.array([80, 82, 81, 79, 80])
print(np.std(group_a))
# Output: ~1.0

# Scenario B: High variance scores (High Spread)
group_b = np.array([50, 100, 20, 150, 80])
```

```
print(np.std(group_b))
# Output: ~46.0
```

```
1.019803902718557
44.27188724235731
```